FPT SCHOOL OF BUSINESS & TECHNOLOGY

ARCHITECTURE & SYSTEM DESIGN

Nhà C, Tòa Nhà Việt Úc, Lưu Hữu Phước,
Quận Nam Từ Liêm, Hà Nội

FPT SCHOOL OF BUSINESS & TECHNOLOGY

AGENDA

- Introduction to ML system architecture
- ML pipeline architecture patterns
- Monolithic and Microservice for ML
- Data flow architecture: Batch vs Stream
- Scalability consideration
- Infrastructure choices
- Design patterns for ML systems
- Uber's Michelangelo platform
- Spotify's event delivery

FPT SCHOOL OF BUSINESS & TECHNOLOGY

Learning Objectives

- Design ML system architecture for different use cases
- Select appropriate architectural patterns for ML systems
- Make informed decisions about infrastructure choices
- Analyze trade-offs between architectural choices
- Design for scalability and maintainability
- Document architecture decisions with justification

FPT SCHOOL OF BUSINESS & TECHNOLOGY

Introduction to ML system architecture

Why Architecture Matters for ML Systems

Architecture is the foundation that determines whether your ML system succeeds or fails in production

Key Statistics:

- 85% of ML projects never make it to production
- Architecture issues account for 40% of production failures
- Poorly designed systems cost 10x more to fix later

The Architecture Challenge: ML systems combine traditional software engineering with unique ML requirements:

- Data pipelines that change
- Models that degrade over time
- Predictions that need real-time serving
- Experiments that need to run in parallel

ML System vs Traditional Software System

TRADITIONAL SOFTWARE

Code → Build → Test → Deploy → Monitor

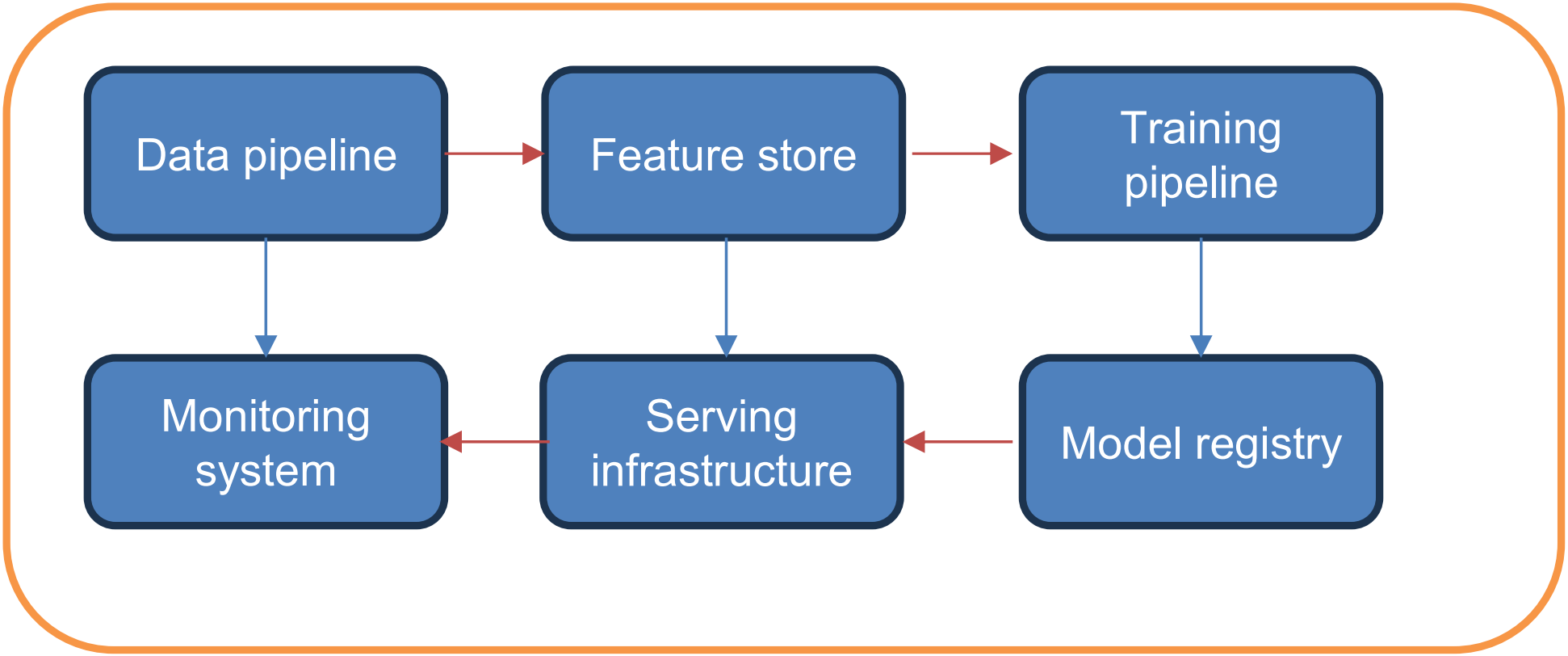
- Deterministic behavior
- Clear specifications
- Predictable outputs

ML Systems

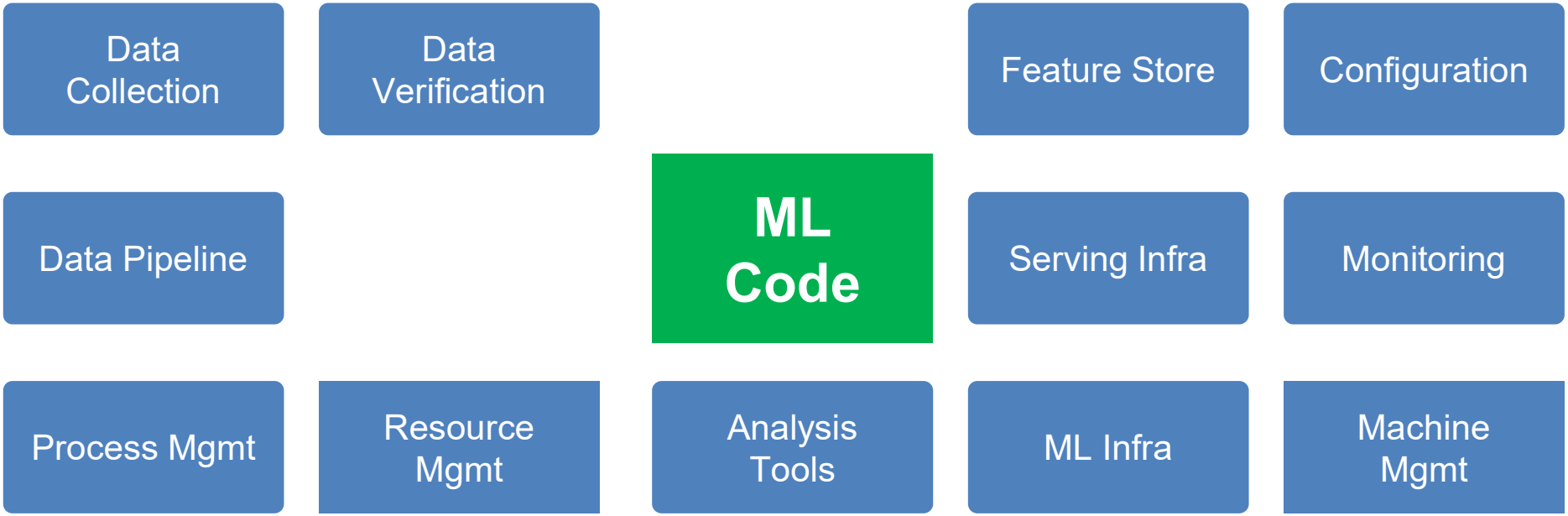
Data + Code + Model → Build → Test → Deploy → Monitor → Retrain

- Probabilistic behavior
- Learned specifications
- Variable outputs
- Continuous data dependencies

Components of a Production ML System



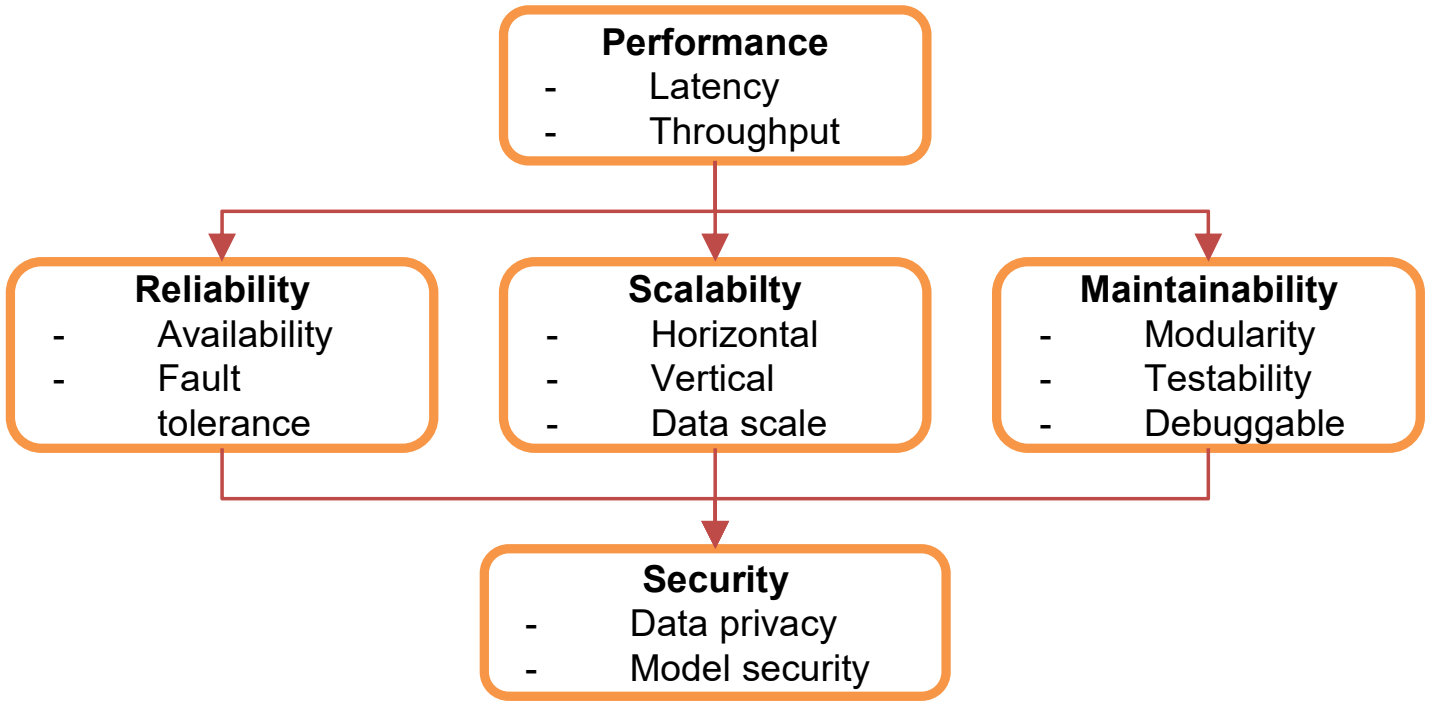
The Hidden Complexity of ML systems



Source: Sculley et al., Hidden Technical Debt in ML Systems, NeurIPS 2015

Architectural Quality Attributes for ML Systems

The "ilities" of ML Systems



Trade-off triangle:

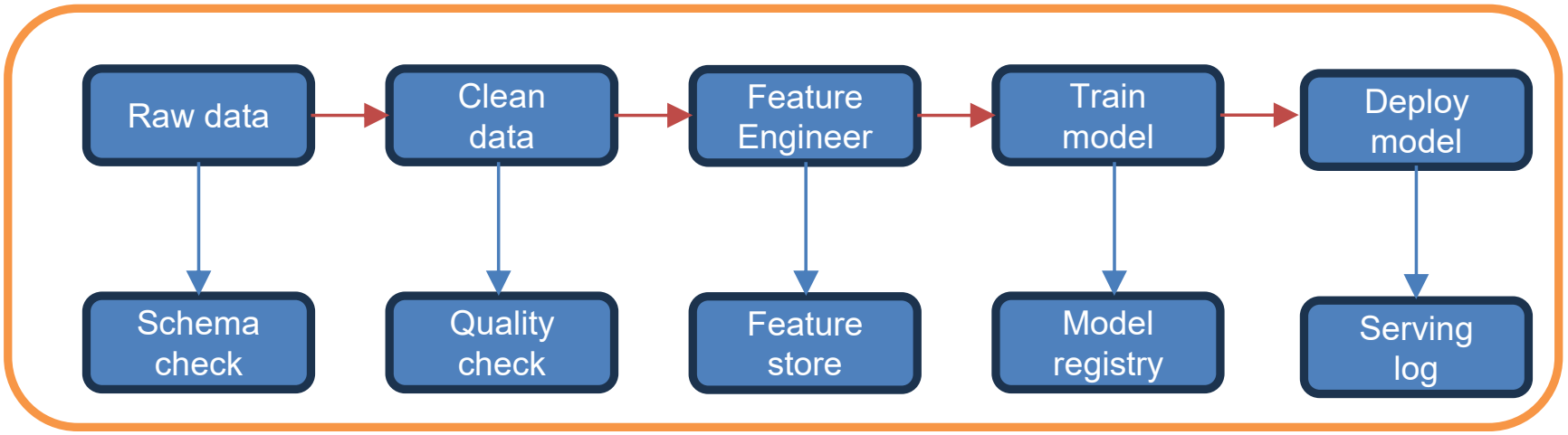
- Cost vs Performance vs Reliability
- Can optimize for 2 but rarely all 3

FPT SCHOOL OF BUSINESS & TECHNOLOGY

ML Pipeline architecture patterns

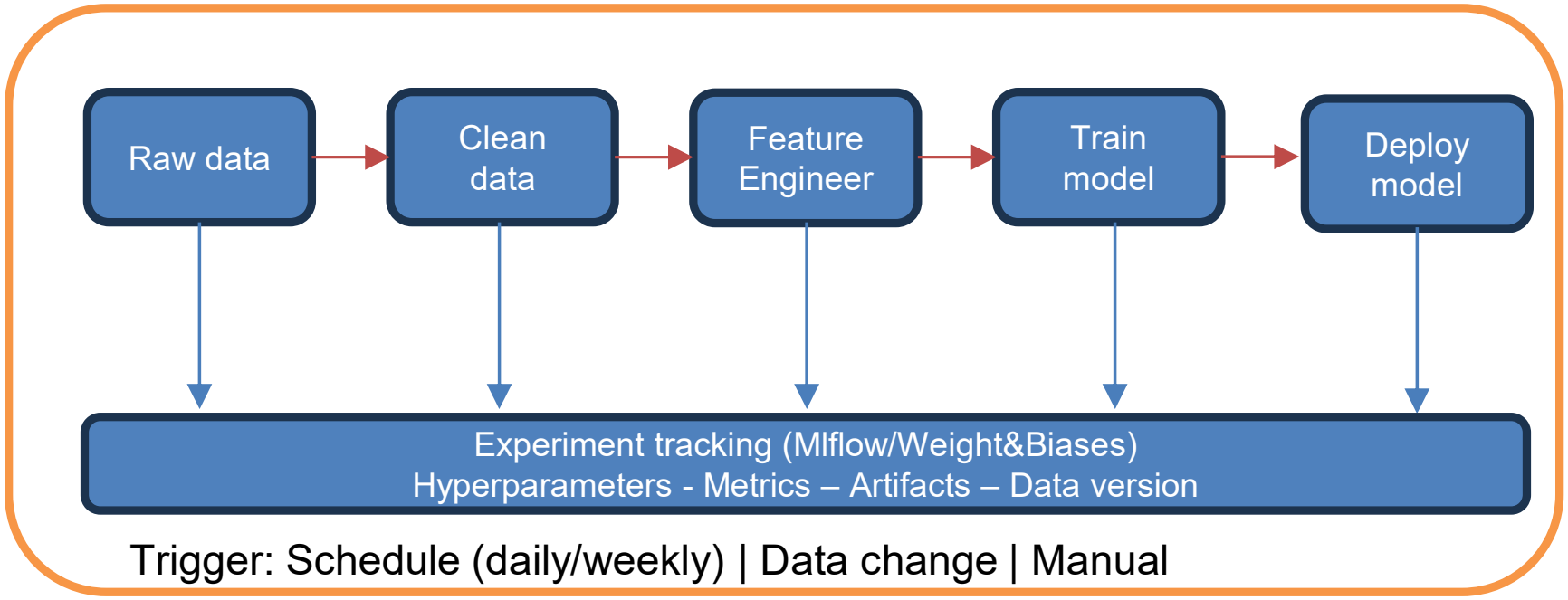
What is an ML Pipeline

Definition: A pipeline is a sequence of data processing steps that transform raw data into predictions

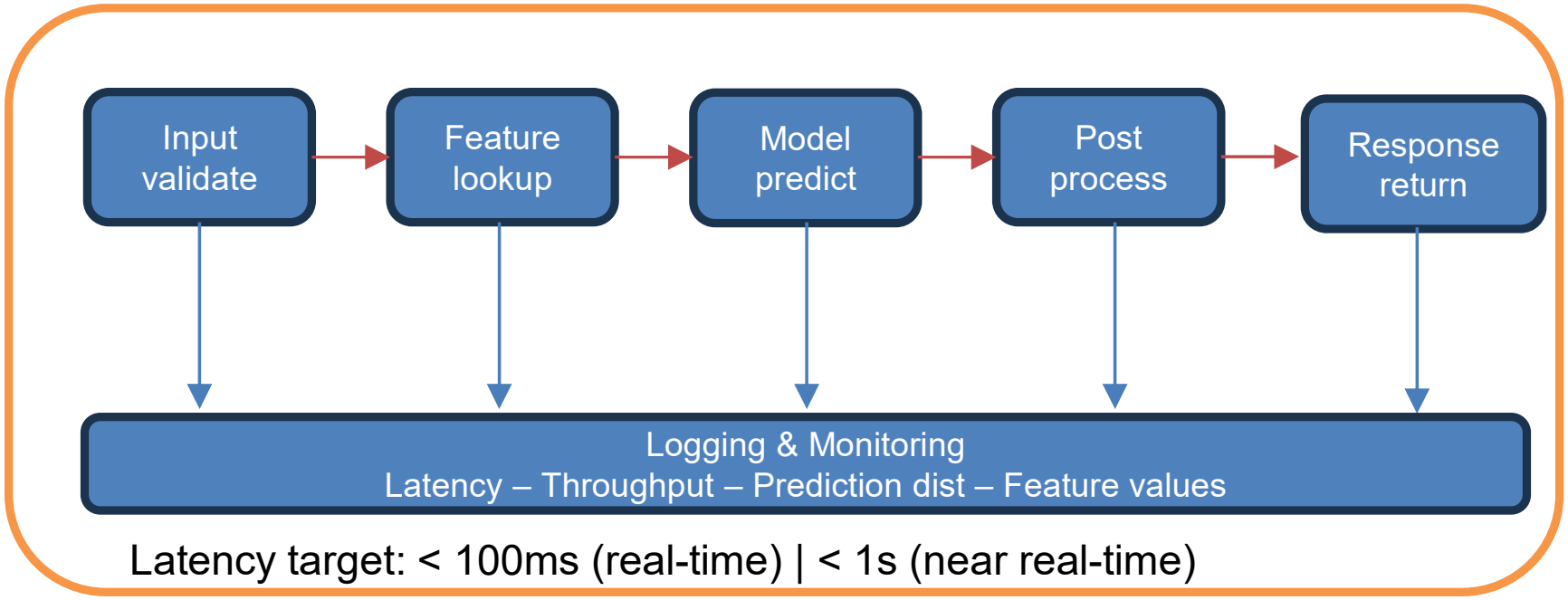


Pipeline Properties:
Reproducibility: Same input → Same output
Modularity: Independent, replaceable steps
Observability: Logging and monitoring at each step
Automation: Triggered by events or schedules

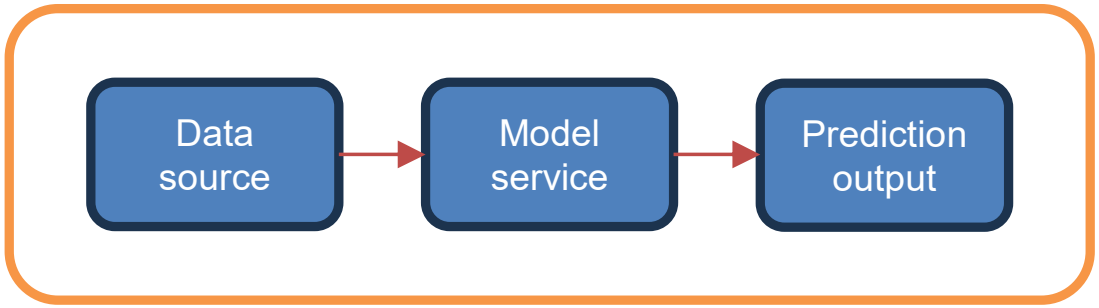
Training pipeline architecture



Inference pipeline architecture



Pattern 1: Simple pipeline (Single model)



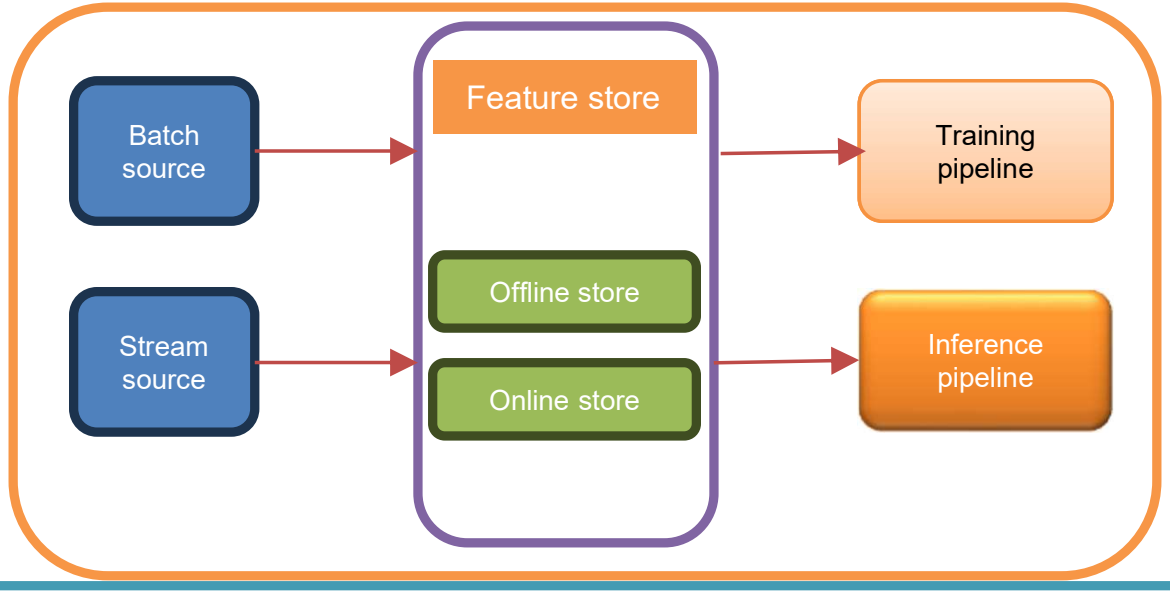
WHEN TO USE:

- Single use case
- Small to medium scale
- Simple feature engineering
- Fast iteration needed

WHEN TO AVOID:

- Multiple models needed
- Complex feature sharing
- High scalability requirements
- Real-time feature computation

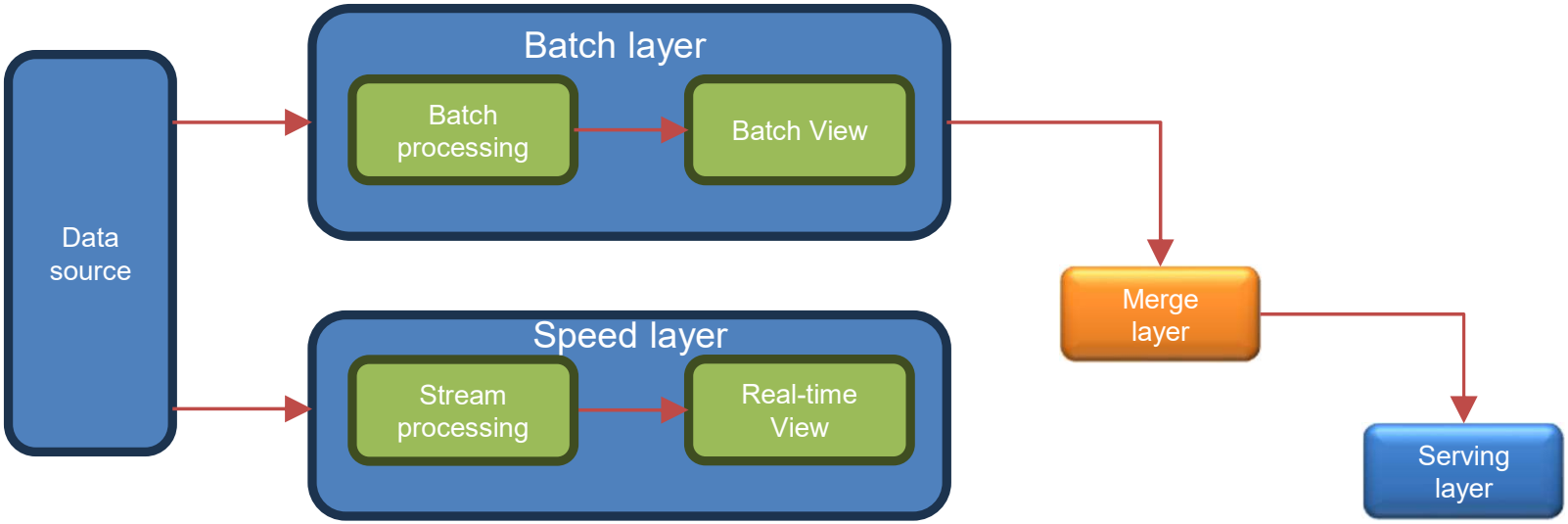
Pattern 2: Feature store patterns



BENEFITS:

- Feature reuse across models
- Consistency between training and serving
- Reduced training-serving skew
- Centralized feature governance

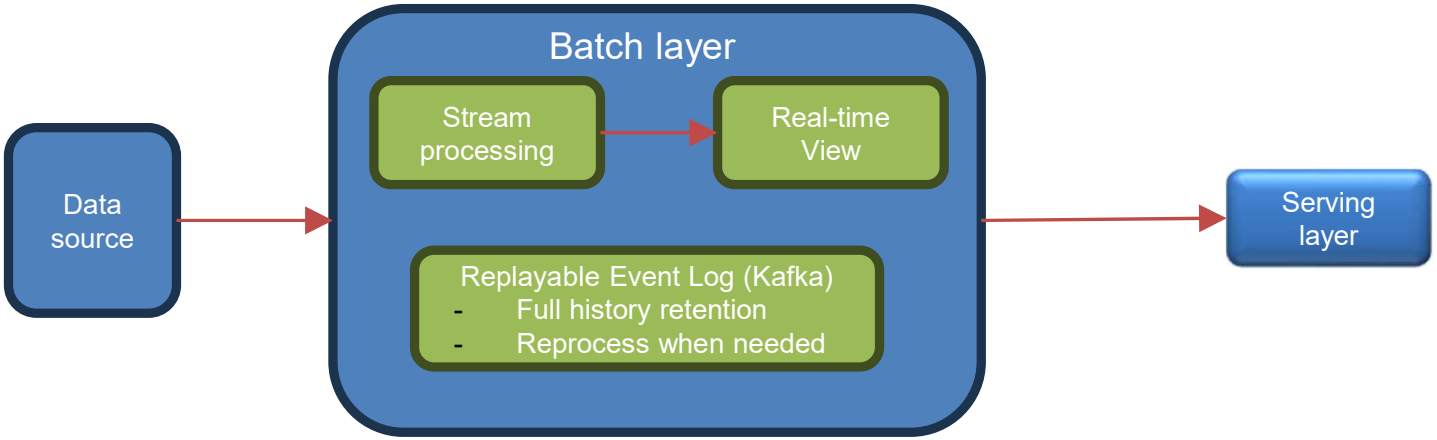
Pattern 3: Lambda architecture



TRADE-OFFS:

- High accuracy (batch) + Low latency (stream)
- Code duplication (2 processing paths)
- Complex to maintain
- Higher operational cost

Pattern 4: Kappa architecture



TRADE-OFFS:

- High accuracy (batch) + Low latency (stream)
- Code duplication (2 processing paths)
- Complex to maintain
- Higher operational cost

Lambda vs Kappa – When to use

Factor	Lambda	Kappa
Query complexity	Complex aggregations	Simpler queries
Data Freshness	Eventual consistency OK	Real-time required
Team Size	Large team	Small team
Development Speed	Slower	Faster
Cost	Higher	Lower
Fault Tolerance	Better	Good

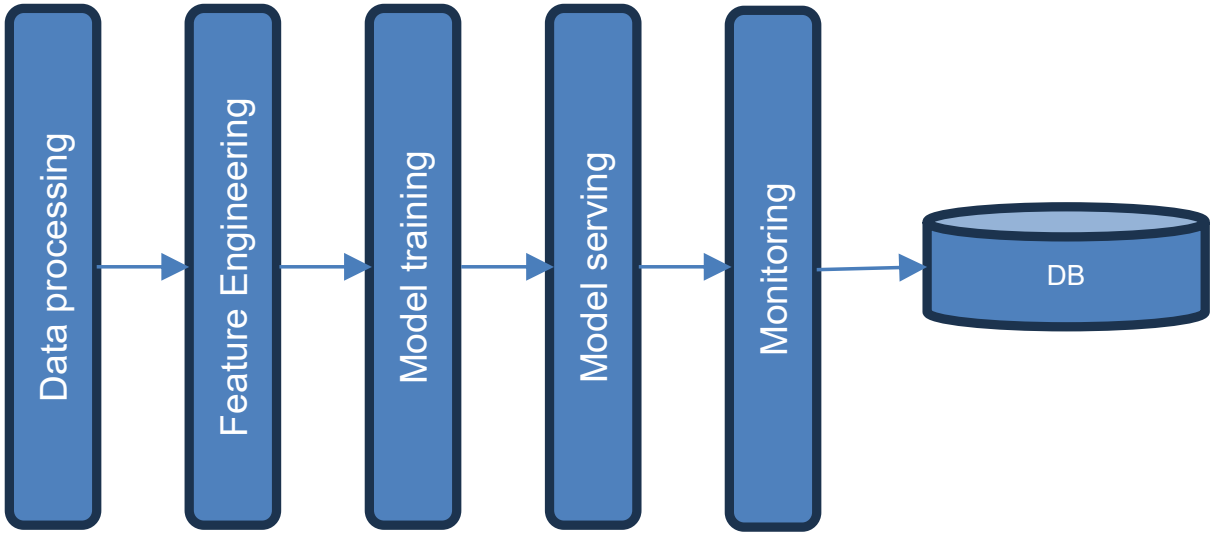
Rule of thumb:

- Start with Kappa for simplicity
- Move to Lambda when you need:
  - Complex historical queries
  - Very large scale batch processing
  - Strict consistency requirements

FPT SCHOOL OF BUSINESS & TECHNOLOGY

Monolithic vs microservice for ML

Monolithic ML architecture



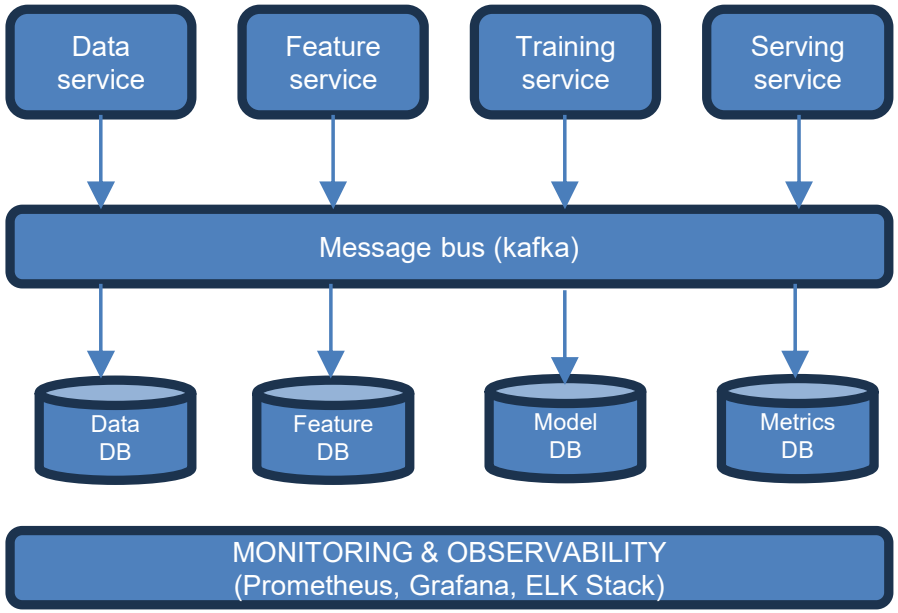
Pros:

- Simple to develop
- Easy to test
- Low latency (in-process)
- Simple deployment

Cons:

- Hard to scale components
- Single point of failure
- Technology lock-in
- Long deployment cycle

Microservice ML architecture



Pros:

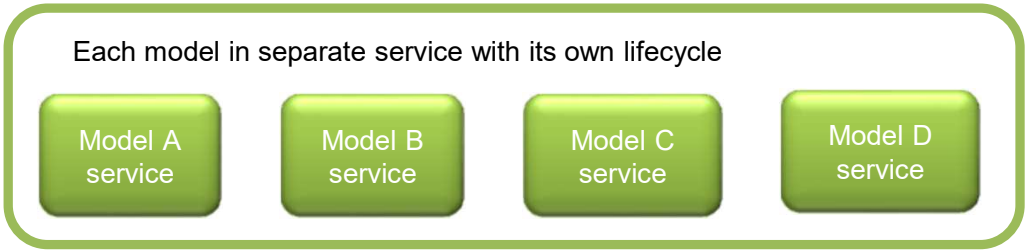
- Independent scaling
- Technology flexibility
- Fault isolation
- Faster deployments

Cons:

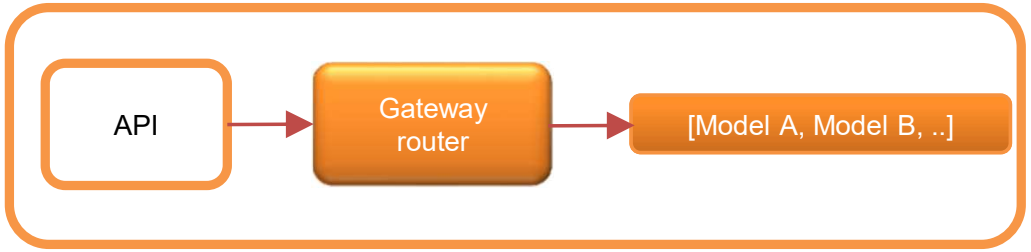
- Network latency
- Complex debugging
- Data consistency challenges
- Operational overhead

ML-Specific microservices patterns

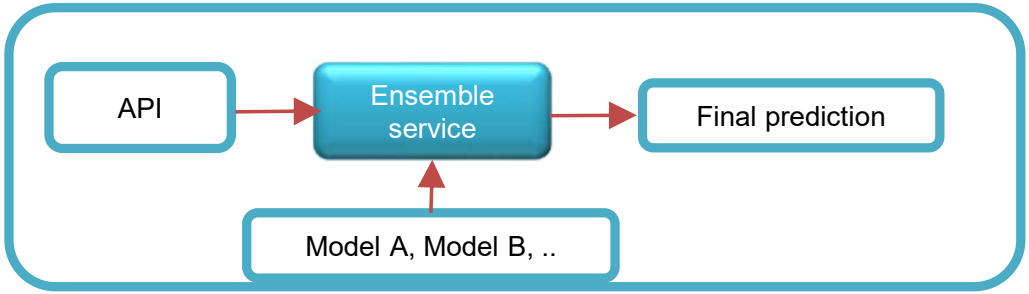
Pattern 1: Model-as-a-service



Pattern 2: Gateway + Model pool



Pattern 3: Ensemble service



Decision Framework – Monolithic vs Microservices

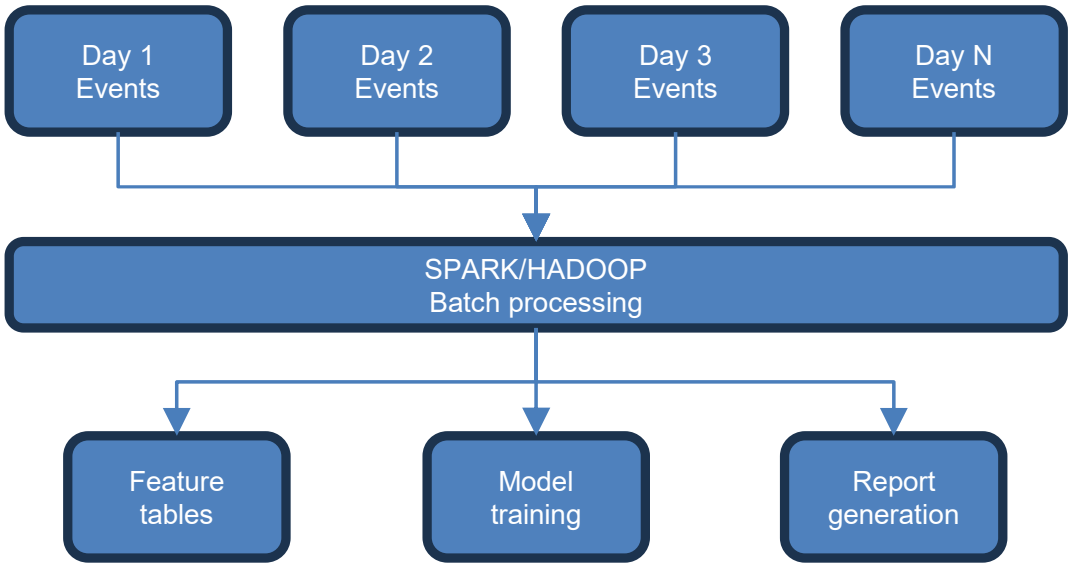
Start with MONOLITHIC when:	Team < 5 engineers
	Single use case / model
	Need fast iteration
	Low traffic volume
	Unclear requirements
Move to MICROSERVICES when:	Multiple teams working on different models
	Different scaling requirements per component
	Need independent deployment cycles
	High availability requirements
	Clear service boundaries
HYBRID APPROACH (Most Common):	Monolithic training pipeline
	Microservices for serving
	Shared feature store

FPT SCHOOL OF BUSINESS & TECHNOLOGY

Data flow architecture
Batch vs Stream

Batch processing for ML

Data lake (HDFS/S3)

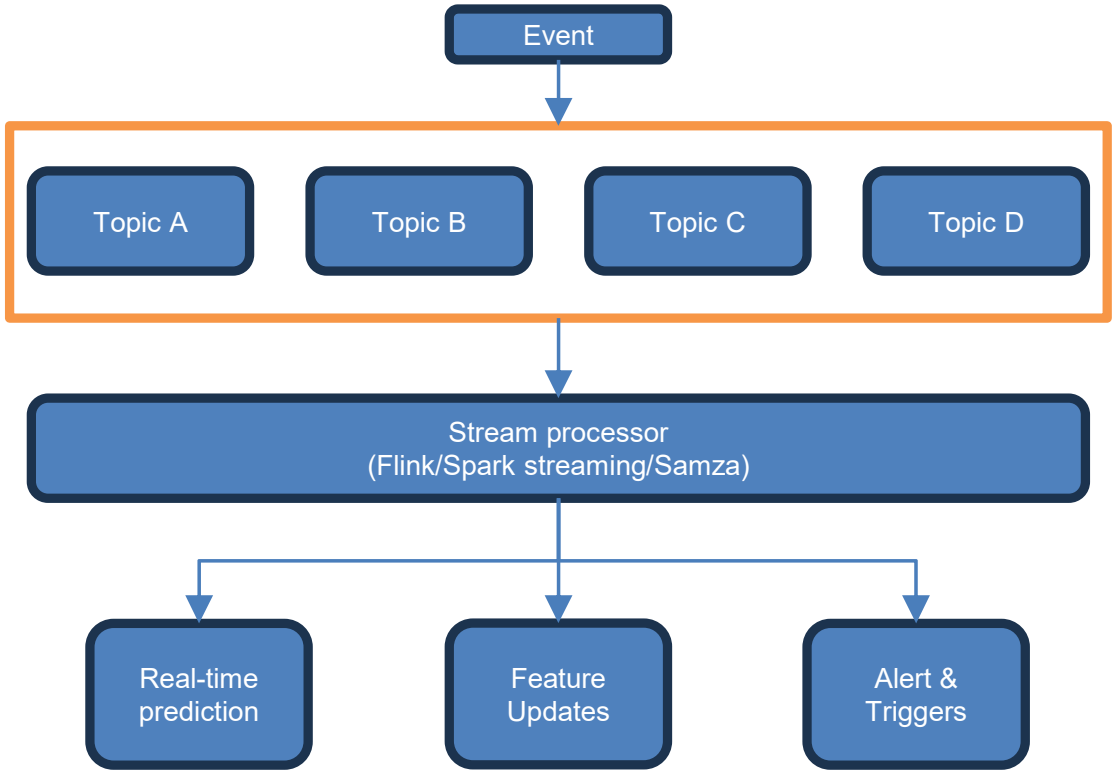


Schedule: Daily/Weekly/Monthly
Latency: Hours to day
Volume: Terabytes to Petabytes

USE CASES:

- Model training and retraining
- Historical feature computation
- Batch predictions (email campaigns)
- Data quality validation - Report generation

Stream processing for ML



Latency: Miliseconds to seconds
Volume: Milions of events per second

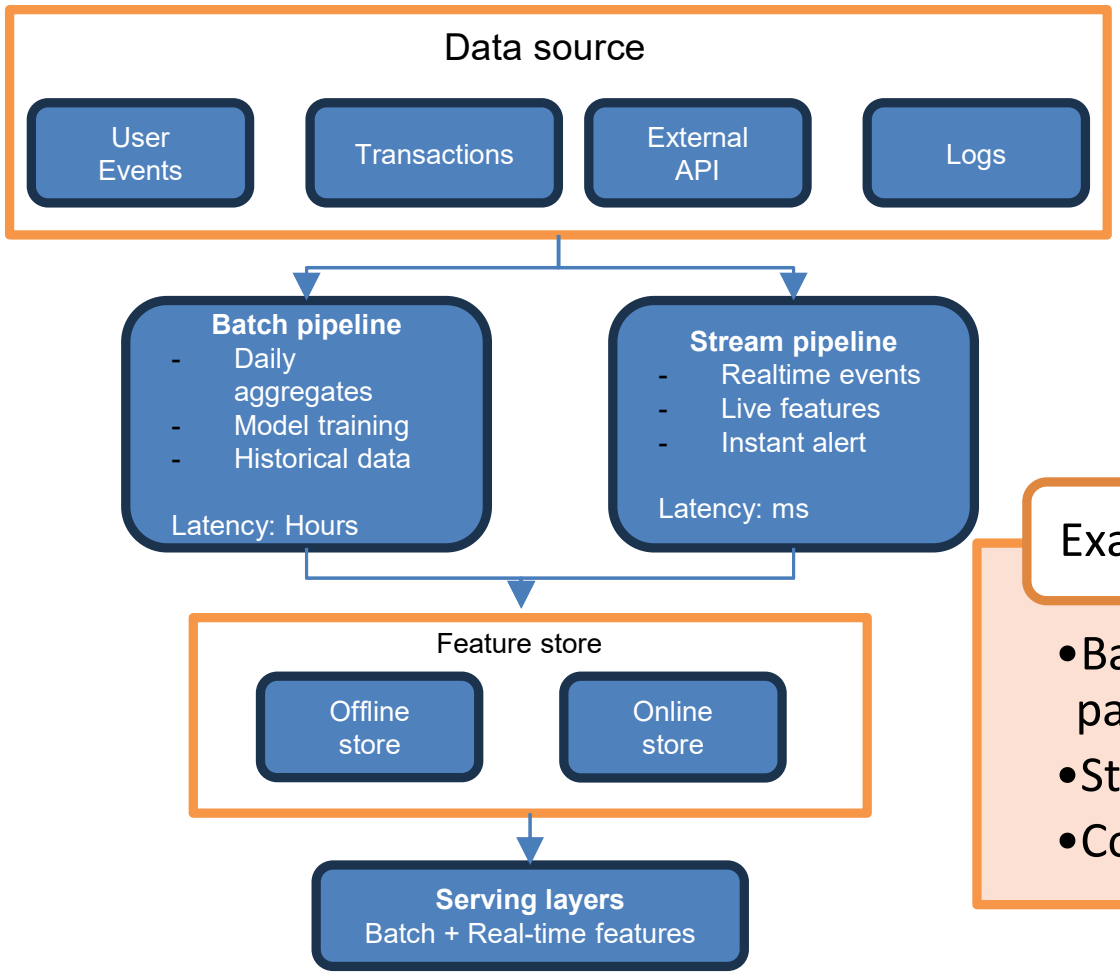
USE CASES:

- Real-time recommendations
- Fraud detection
- Live feature computation
- Model monitoring & alerting
- Real-time personalization

Batch vs Stream – Trade-offs

Dimension	BATCH	STREAM
Latency	Hours/Days	Miliseconds/seconds
Throughput	Very high	High
Cost	Lower	Higher
Complexity	Easy (reprocess)	Complex (checkpoints)
Fault Recovery		
Data Completeness	Complete	May miss events
State Management	Stateless	Stateful

Hybrid batch-stream architecture



Example: Fraud detection

- Batch: User's historical spending pattern
- Stream: Current transaction context
- Combined: Real-time fraud scoring

Feature freshness decision matrix

Feature	Computation complexity	Update Frequency	Use cases
Static Features	Low	Weekly/Monthly	User demographics, movie metadata
Batch features	Medium-high	Hourly/Daily	User watch history, 7-day aggregates
Near real-time	Medium	Minutes	Recent activity, Session context
Real-time features	Low	Seconds/Miliseconds	Current location, Device info

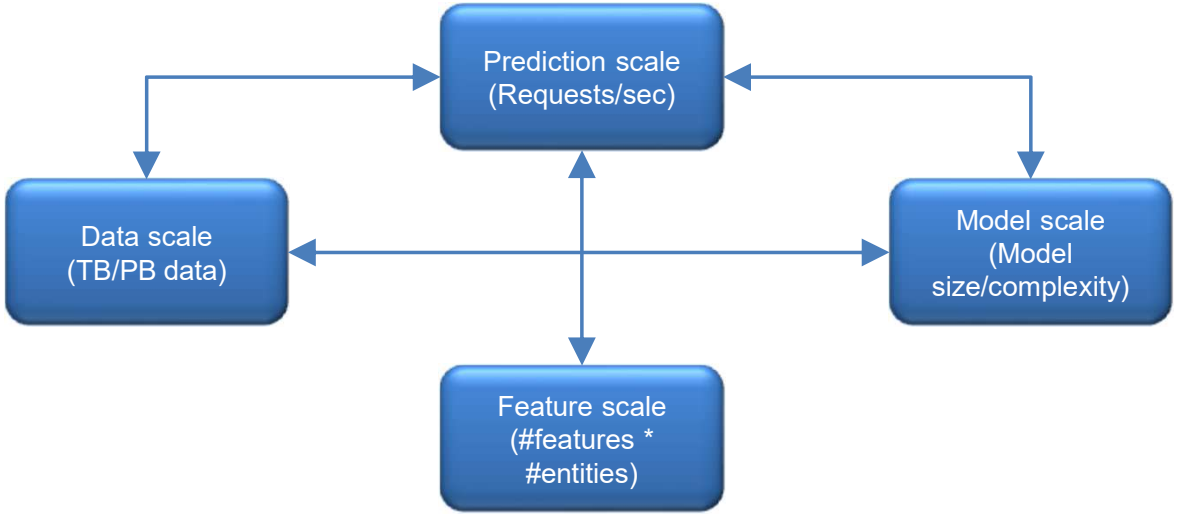
KEY INSIGHT:

- Match feature freshness to business value
- More fresh = More compute cost
- Only pay for freshness you need

FPT SCHOOL OF BUSINESS & TECHNOLOGY

Scalability considerations

Scalability dimension in ML systems

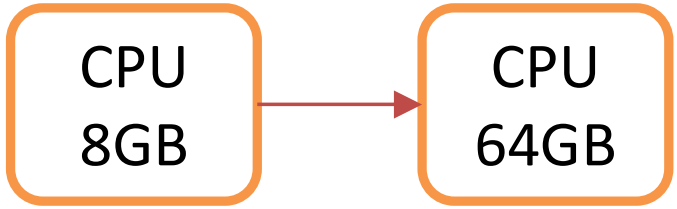


Each dimension requires different scaling strategies

Horizontal vs vertical scaling

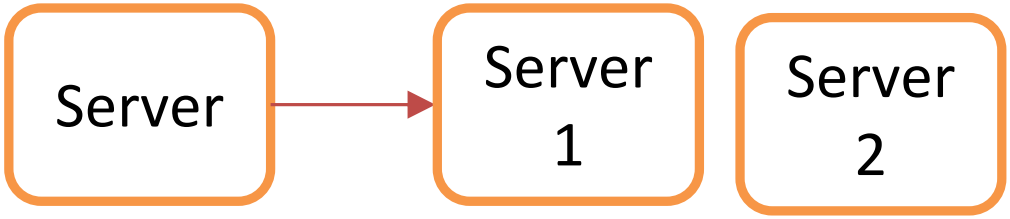
Vertical scaling (scale up)

- Simple to implement
- No code change
  - Physical limits exist
  - Single point of failure



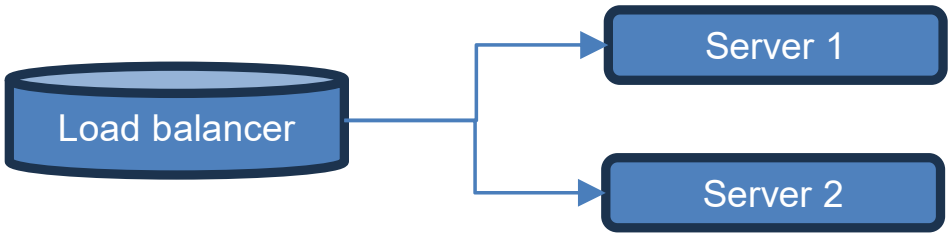
Horizontal scaling (Scale out)

- Nearly unlimited scale
- Better fault tolerance
  - More complex architecture
  - Data consistency challenges

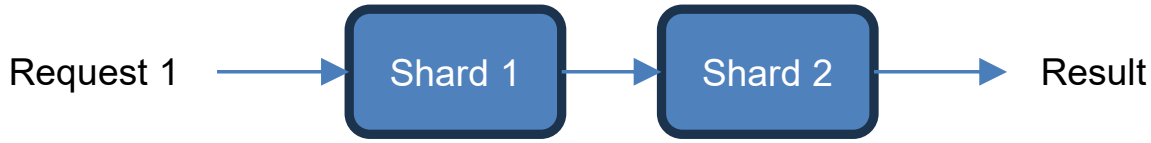


Scaling ML serving

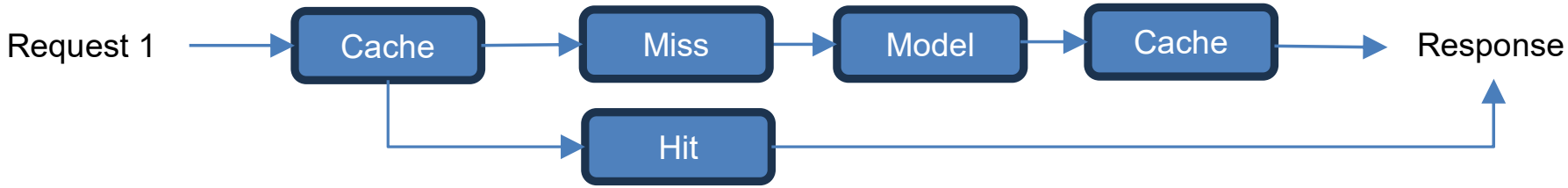
Pattern 1: Replicate model servers



Pattern 2: Model sharding (large models)
Use for: large embedding tables, LLMs



Pattern 2: Model sharding (large models)
Use for: large embedding tables, LLMs



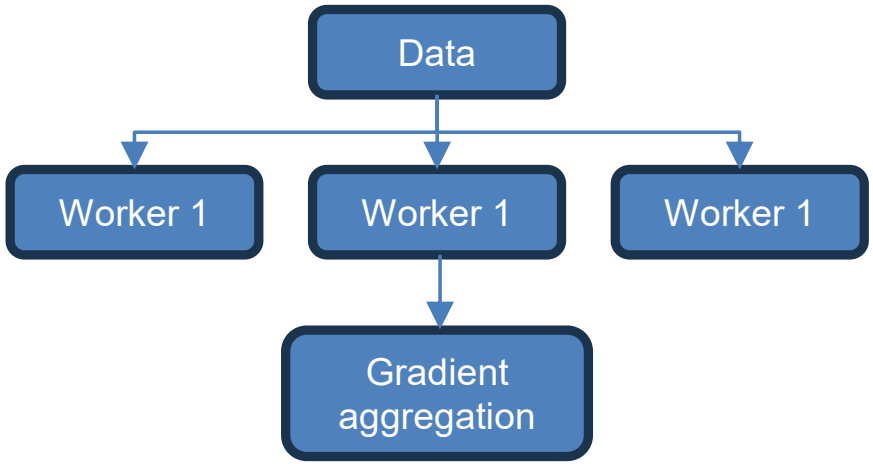
Scaling ML training

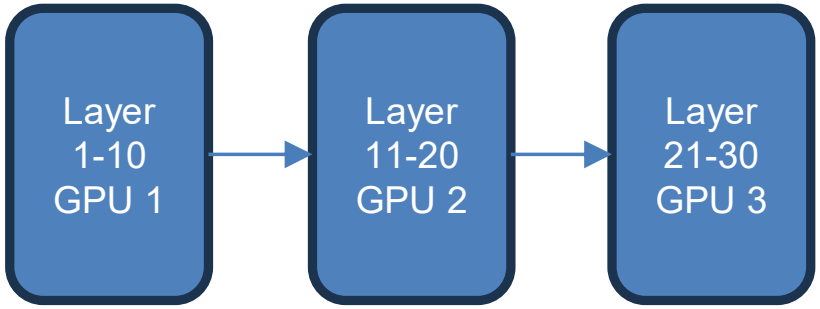
Data parallelism

- Split data across multiple workers, each has full model copy

Model Parallelism (for very large models)

- Split model across multiple workers
- Use for: Transformer, LLM (GPT, BERT)





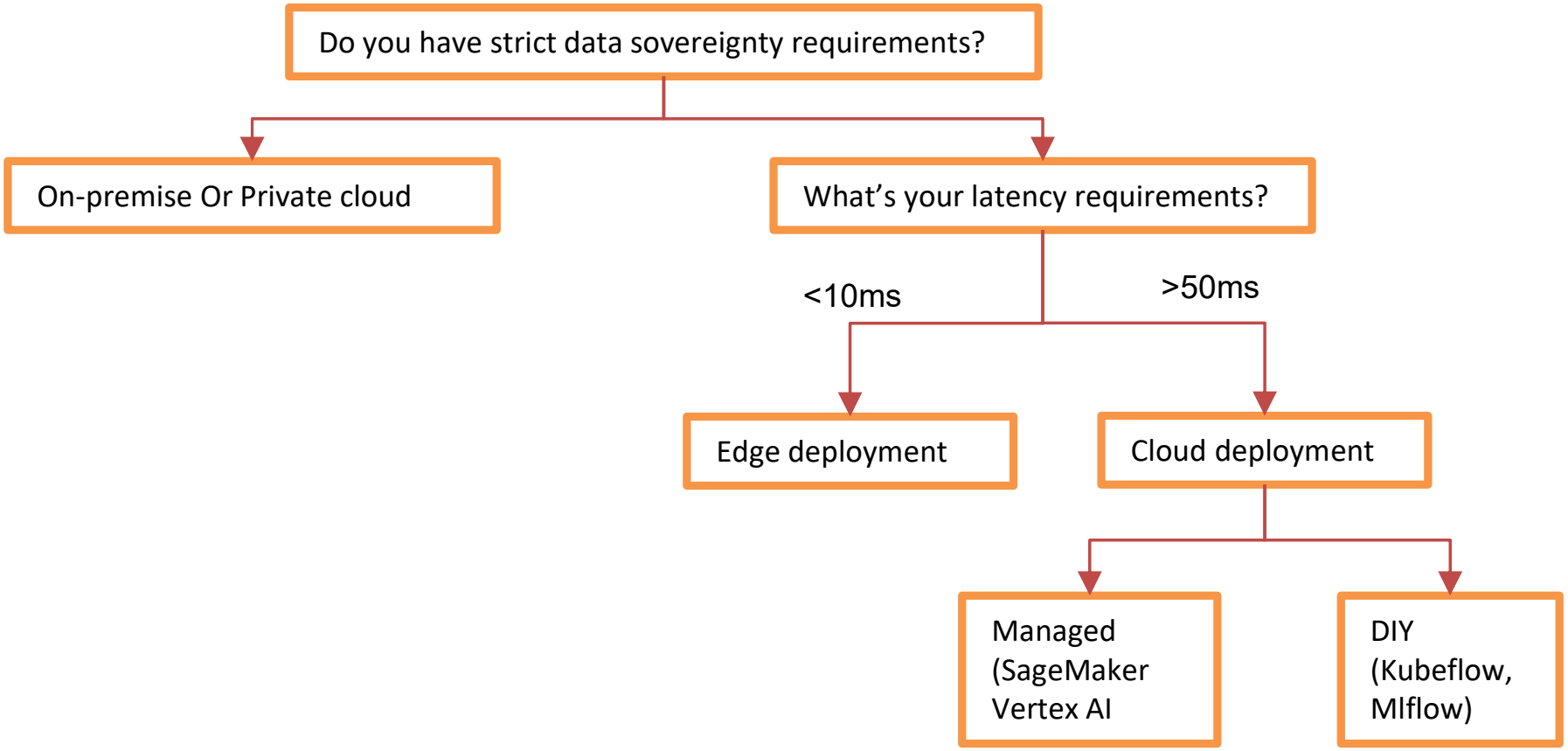
Cloud vs On-Premise vs Hybird

Cloud	
Elastic scaling Managed services	Pay-as-you-go Quick to start
On-Premis	
Data center: Full control No vendor lock-in	*Hardware*: Data sovereignty Predictable cost
Hybrid	
On-premise For training Complex networking	*Cloud for serving* Data sync challenges

Edge vs Cloud deployment decision

	Cloud	Edge
Latency	*50-500ms*	<10ms
Connectivity	Required	Optional
Compute	Unlimited	Limited
Privacy	Data leaves	Data stays
Cost	Pay-as-you-go	Upfront + ops
Update	Easy	Complex
Model size	Any	Constrained

Infrastructure decision framework



FPT SCHOOL OF BUSINESS & TECHNOLOGY

Design patterns for ML systems

Pattern 1 – Model as service


```
POST /predict {
   "features": [1, 2, 3]
}
```

Model service

Trained model
(loaded in mem)

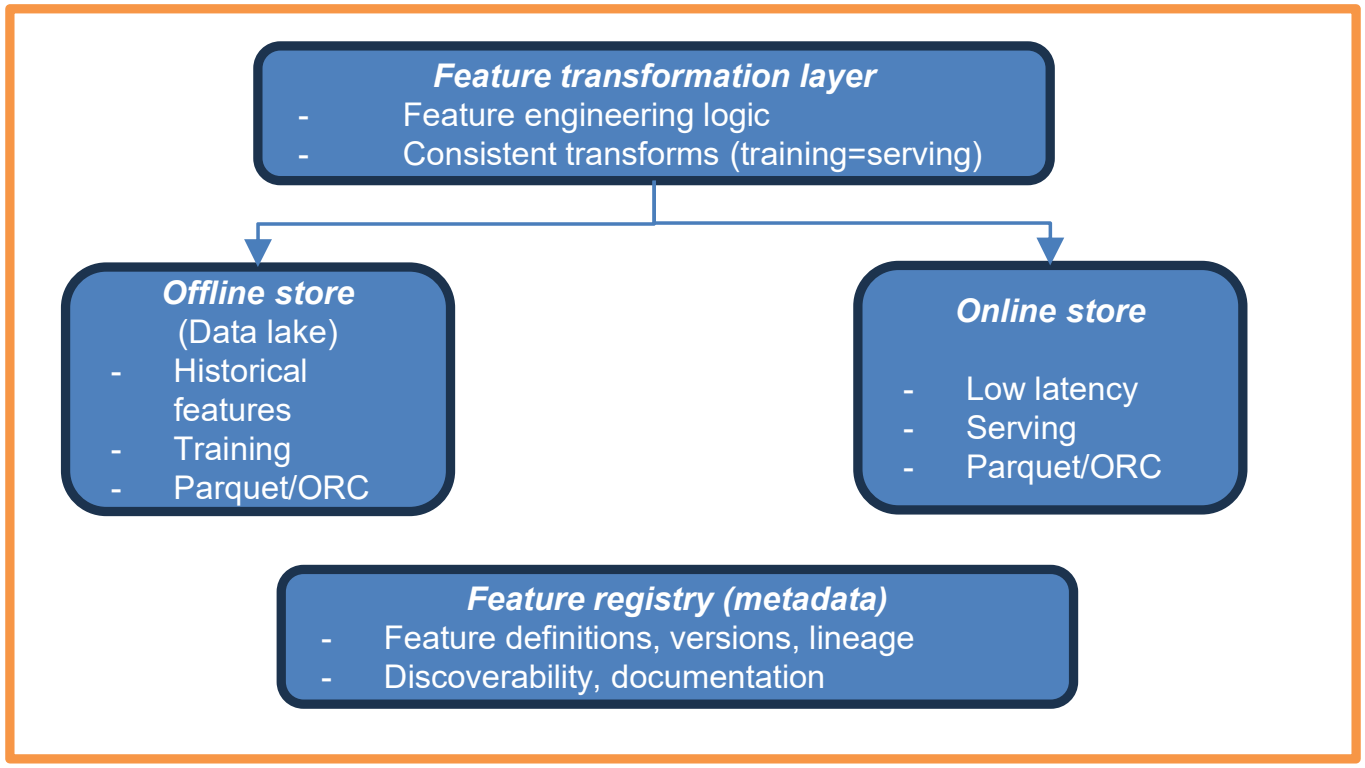
Responsibilities:
- Load model on startup
- Preprocess input
- Run inference
- Post-process
- Log predictions

```
{
   "prediction": 0.87,
   "condidence": 0.95
}
```

Implementation options:

- Flask/FastAPI + pickle
- Tensorflow Serving
- TorchServing
- BentoML
- Seldon Core

Pattern 2 – Feature store pattern



Tools: Feast, Hopsworks, Tecton

Pattern 3 – Model registry pattern

Model lifecycle:
Train → Register → Validate → Stage → Deploy → Monitor

(All tracked in registry)

Tools: Mlflow model registry, SageMaker Model Registry

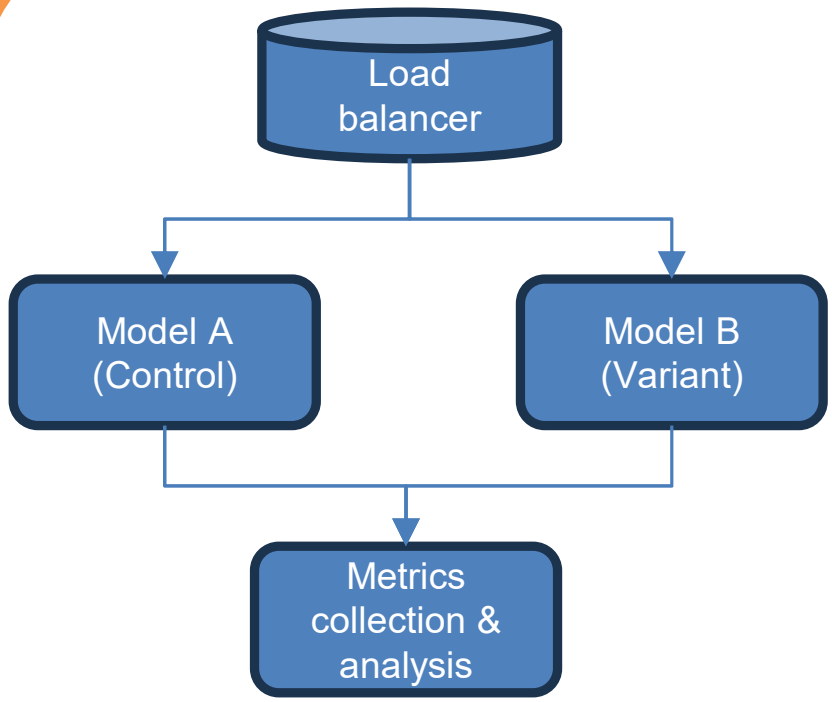
Model registry

Model artifacts
- model.pkl/.pt
- Config.yaml
- Requirements.txt

Meta data
- Training data version
- Hyper-parameters
- Metrics (AUC, F1)
- Author, timestamp

State management
- None → stagging → Prod

Pattern 4 – A/B Testing & Canary deployment



A/B Testing (Feature/Model experimentation)

Phase 1:
OLD model – 95%
New model – 5%

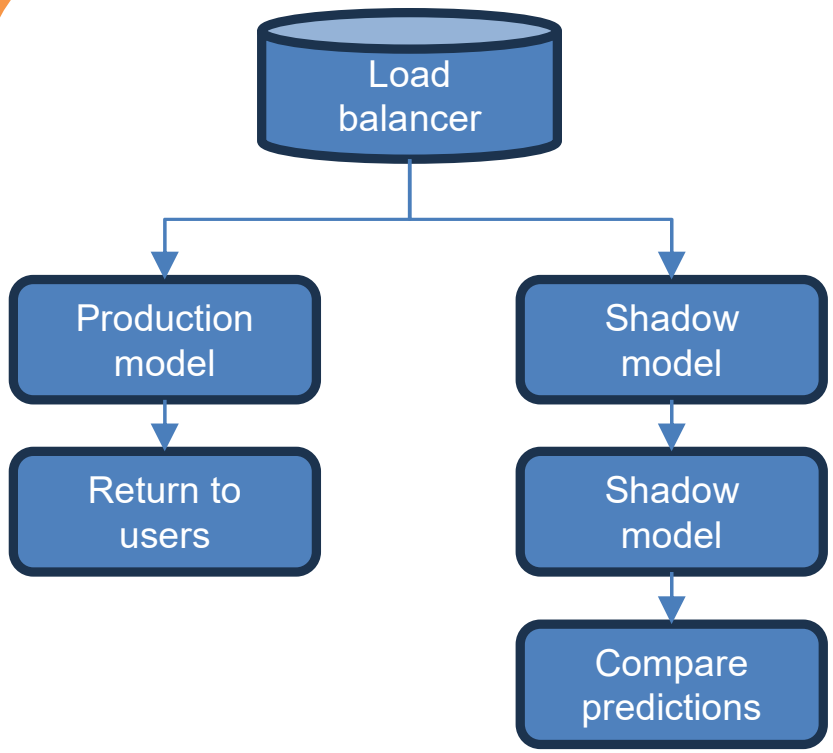
Phase 2:
OLD model – 75%
New model – 25%

Phase 3:
OLD model – 50%
New model – 50%

Phase 4:
New model – 100%

Canary deployment

Pattern 5 – Shadow mode/Dark launch



Purpose: Test new model without impacting users

FPT SCHOOL OF BUSINESS & TECHNOLOGY

Uber’s Michelangelo platform

Uber Before Michelangelo

The Problems (Pre-2015)

✕ TOOL FRAGMENTATION

- Data scientists used R, scikit-learn, custom tools
- No standard way to deploy models
- Each team built bespoke deployment systems

✕ NO STANDARDIZATION

- No reliable, reproducible pipelines
- Inconsistent data processing
- Training-serving skew was common

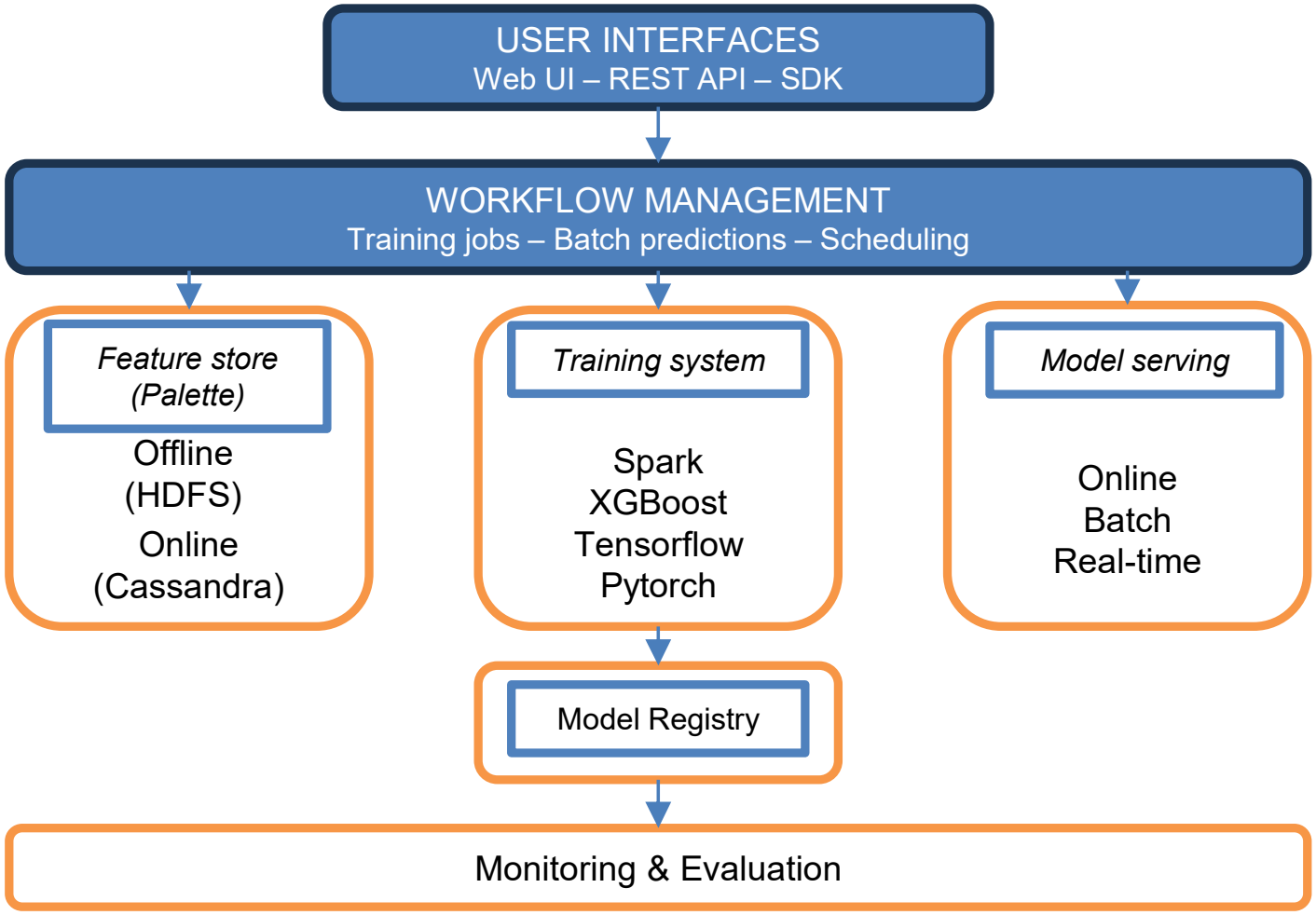
✕ SCALABILITY ISSUES

- Model training limited to desktop-sized data
- No central storage for experiment results
- Difficult to compare models

✕ LIMITED IMPACT

- Huge resources needed to translate local model to production
- Unreliable ML and data pipelines
- Engineering teams built custom serving systems

Michelangelo Architecture Overview



Michelangelo Feature Store (Palette)

"Feature management is the biggest bottleneck in productizing ML "
- Uber Engineering

BATCH FEATURES (Spark)

- sum_orders_7day
- avg_order_size_30day
- restaurant_avg_prep_time_7day

NEAR REAL-TIME FEATURES (Kafka + Flink)

- orders_last_5min
- eyeball_impressions_5min
- current_queue_length

RPC FEATURES (External APIs)

- weather_conditions
- traffic_estimate
- third_party_signals

Michelangelo Training-Serving Consistency

The Training-Serving Skew Problem:

The problems:

- Training time is different serving time

Solutions: Domain specific language (DSL)

- Feature transforms difined ONCE and applied EVERYWHERE
- Training pipeline and serving pipeling have the same DSL

Results

- Feature parity guaranteed by design

Michelangelo - Results & Lessons

Business impact:

- Hundreds of ML models in production
- Powers: ETAs, Pricing, Fraud, UberEats, Driver Dispatch
- Models deployed in hours instead of months
- Democratized ML across engineering teams
- 10x increase in ML experiments per month

Key architecture decisions:

- Centralized Feature Store (Palette) → Enabled feature reuse, reduced duplication
- Unified Offline/Online Pipelines → Eliminated training-serving skew
- Model Repository with Full Lineage → Reproducibility, debugging, compliance
- Standardized but Flexible → Support multiple frameworks (XGBoost, TensorFlow, PyTorch)

FPT SCHOOL OF BUSINESS & TECHNOLOGY

Spotify’s event delivery

Spotify's Data Scale

6,000,000	600M+	1+ Trillion	180+
Daily streams	Monthly active users	Events/day collected	Countries

Every user action generates an event:

- Play songs
- Skip track
- Like/Dislike
- Create playlist
- Search query
- Download

Challenge: Process ALL events reliably with predictable latency

Spotify Event Delivery - Evolution

Generation 1: Syslog + file copy

Client → Syslog → Files → scp → HDFS

Problems:

- Components tightly coupled
- Hard to scale
- Outages became frequent
- Small changes caused system failures

Generation 2: Kafka-based

Client → Kafka → Stream → HDFS

Challenges:

- Operating Kafka at Spotify scale
- Data loss during failures
- Required significant ops investment

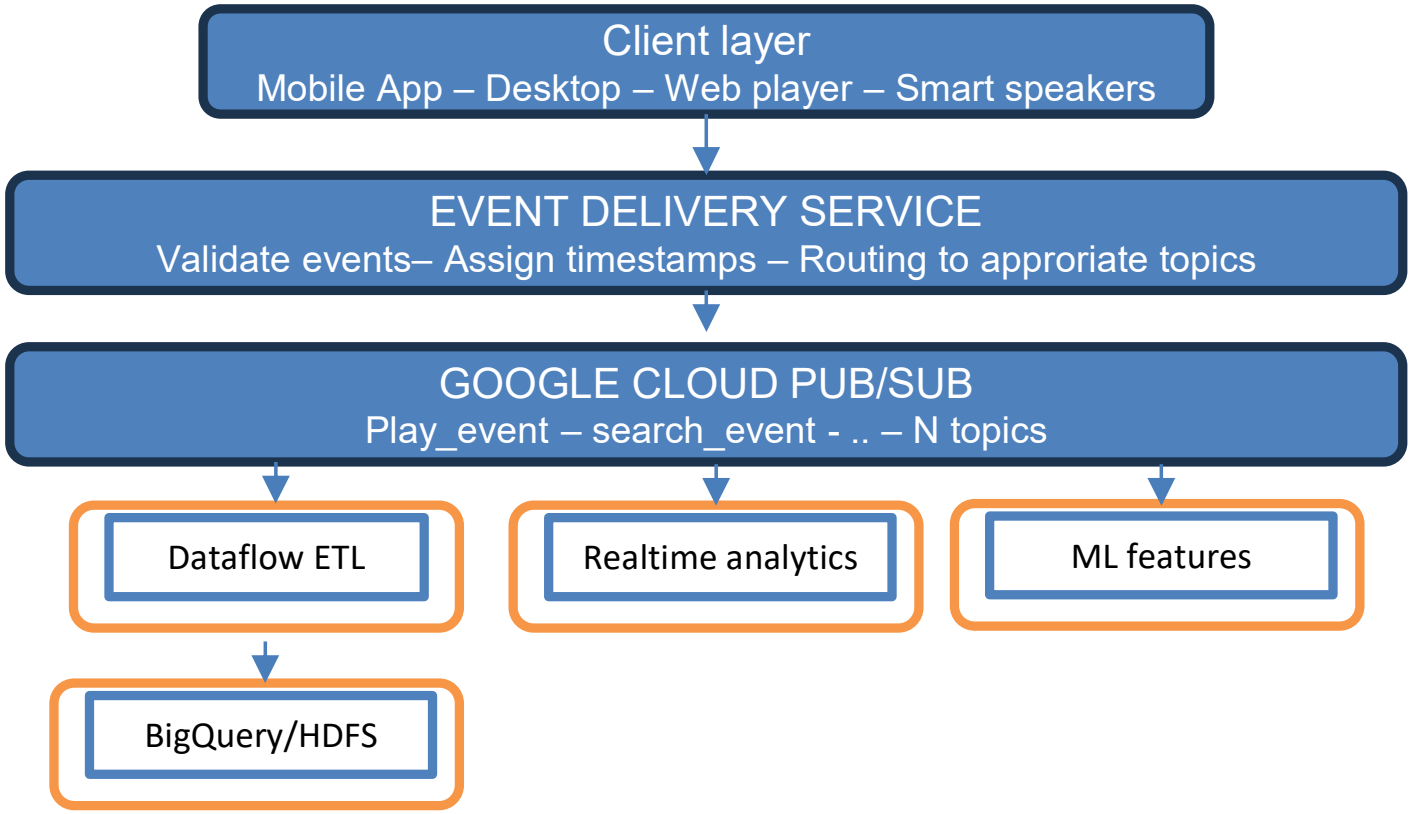
Generation 3: Google Cloud pub/sub

Client → Cloud Pub/Sub → Dataflow → BigQuery/HDFS

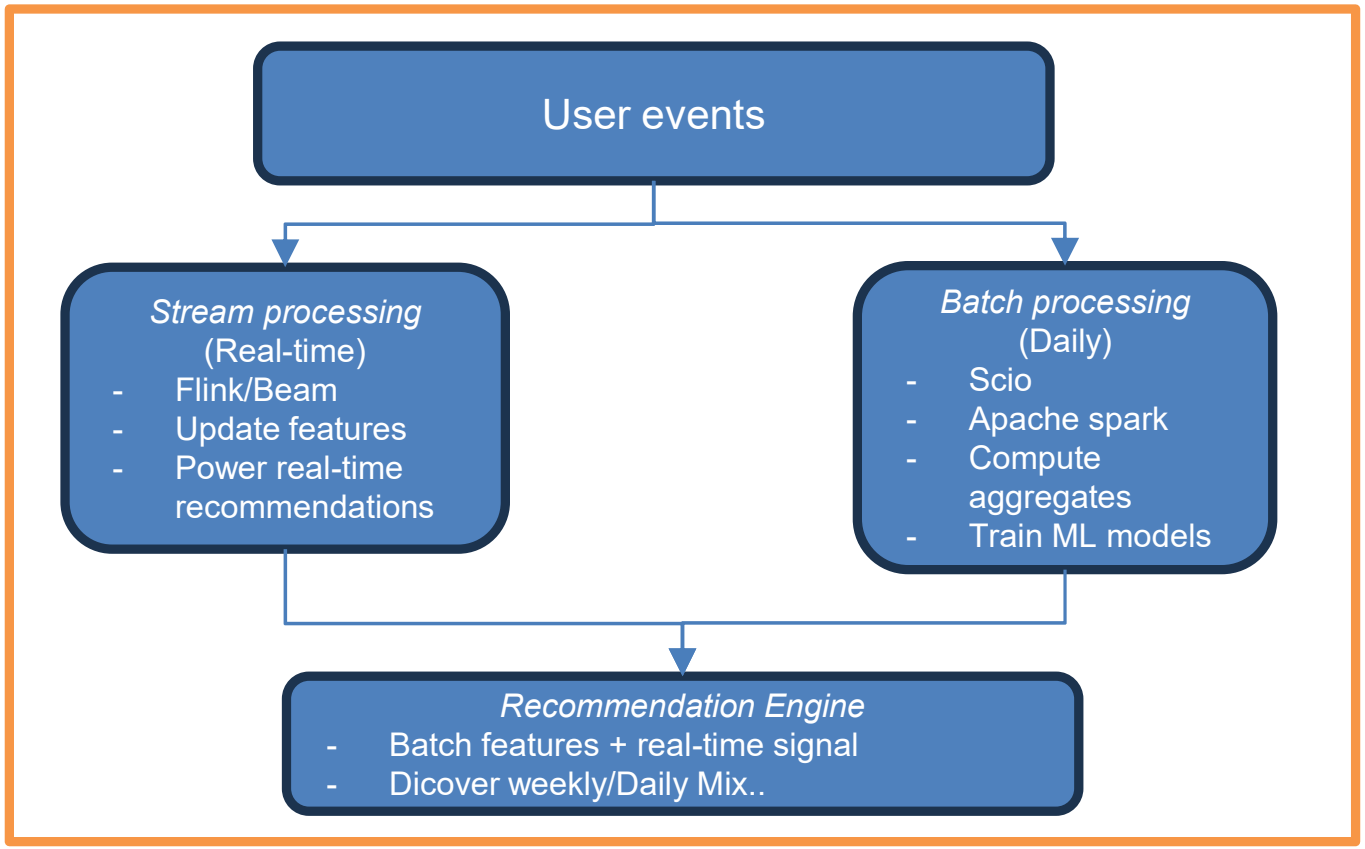
Benefits:

- Managed service (no ops burden)
- Built-in reliability
- Elastic scaling
- Separate topics per event type

Spotify Event Delivery Architecture



Spotify Hybrid Processing Architecture



Lessons Learned

EVENT-DRIVEN ARCHITECTURE

Events are the primary data object
Decouple producers from consumers

EMBRACE HYBRID (Batch + Stream)

Match processing to business requirement
Don't over-engineer for real-time

MANAGED SERVICES FOR CORE INFRAS

Cloud Pub/Sub instead of self-managed Kafka
Focus engineering on business logic, not ops

DATA QUALITY AT THE SOURCE

Validate early, fix at source
Schema enforcement

SEPARATE TOPICS PER EVENT TYPE

Efficient real-time processing
Consumers only subscribe to relevant events

References

[1] Uber: "Meet Michelangelo: Uber's ML Platform", https://www.uber.com/blog/michelangelo-machine-learning-platform/

[2] Spotify: "Event Delivery - The Road to the Cloud", https://engineering.atspotify.com/2016/02/event-delivery-cloud/

FPT SCHOOL OF BUSINESS & TECHNOLOGY

THANK YOU